# Pricing Baku Real Estate from Scratch: Decision Trees and Pegasos SVMs on bina.az Listings

Aghazade, Ibrahim  Aliyev, Sahib  Mammadli, Mahammad  Sakhrayev, Khizri

*MLE-AI-201 Machine Learning — AI Academy, National Artificial Intelligence Center*

***Abstract*—**We predict the prices of real-estate listings scraped from bina.az, using 91,314 cleaned sale listings. Task A regresses the logarithm of the price; Task B classifies each listing as premium or standard, split at the median training price of 215,000 AZN. We implemented two models from scratch in NumPy: a CART-style decision tree with Gini, entropy and MSE criteria, and a Pegasos sub-gradient support-vector machine, either linear or on random Fourier features that approximate an RBF kernel. Both are benchmarked against matched scikit-learn models on identical splits, with 5-fold cross-validation, a held-out test set and timing. On the test set our regression tree reaches $R^2 = 0.853$ on log price, against 0.852 for scikit-learn. Our Gini tree reaches F1 0.926 and ROC-AUC 0.971, and our linear Pegasos SVM, chosen on validation over the Fourier-feature variant, reaches F1 0.904. Each of our models is within 0.007 F1 of its best matched scikit-learn baseline, and our trees are nearly identical to scikit-learn's, but they fit about 277 times more slowly. Errors concentrate near the tier threshold, at both price extremes and in districts with little training data. The main takeaway: careful from-scratch implementations reach library accuracy on this data; what they lose is speed. Code: `github.com/mmahammad57/binaaz-ml-project` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, regression, classification, real-estate price prediction.

## I. Introduction

Listing prices on bina.az, a property portal for Azerbaijan, are set by sellers and agents. A model that predicts a listing's price from its size, type and location helps both sides judge whether an asking price is reasonable. We study two tasks on the same listings. *Task A* is regression of $\log(\text{price})$. *Task B* is classification of a derived *price tier*: premium if the price is at least the median training price, standard otherwise.

Our contributions are:

- a decision tree written from scratch in NumPy, for regression (MSE) and classification (Gini or entropy), with the usual stopping rules and impurity-based feature importances;
- a Pegasos support-vector machine written from scratch, linear or on random Fourier features (RFF) for a non-linear decision boundary;
- a benchmark of both against matched scikit-learn models [5] on identical data: cross-validation, a held-out test set, timing and a scaling study;
- an error analysis by price range, district, category and distance from the tier threshold;
- a reproducible pipeline: one command regenerates every number, table and figure in this report.

## II. Background

### A. Decision trees

A CART tree [1] splits a node $S$ on one feature $j$ and a threshold $t$ into $S_L = \{x \in S : x_j \le t\}$ and $S_R = S \setminus S_L$, choosing the split that most reduces an impurity $I$. For classes $k$ with proportions $p_k$ in $S$, and for a regression target $y$ with mean $\bar{y}_S$,

$$I_{\text{Gini}}(S) = 1 - \textstyle\sum_k p_k^2, \qquad I_{\text{ent}}(S) = -\textstyle\sum_k p_k \log_2 p_k, \qquad (1)$$
$$I_{\text{MSE}}(S) = \tfrac{1}{|S|} \textstyle\sum_{i \in S} (y_i - \bar{y}_S)^2.$$

The impurity decrease, called information gain for entropy, is

$$\Delta(S, j, t) = I(S) - \tfrac{|S_L|}{|S|} I(S_L) - \tfrac{|S_R|}{|S|} I(S_R). \qquad (2)$$

Splitting is repeated on each child until a stopping rule holds; a leaf predicts the class proportions or the mean target of its rows.

### B. Support-vector machines and Pegasos

The soft-margin SVM [2] with labels $y_i \in \{-1, +1\}$ minimises $\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i$ subject to $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \ge 1 - \xi_i$, $\xi_i \ge 0$. Eliminating the slacks gives the hinge-loss primal

$$J(\mathbf{w}, b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n} \max\big(0,\, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)\big), \qquad (3)$$

which has the same minimiser as the constrained form when $\lambda = 1/(nC)$. Pegasos [3] minimises (3) by stochastic sub-gradient steps. At step $t$ it draws an example $i$, sets $\eta_t = 1/(\lambda t)$ and updates

$$\mathbf{w} \leftarrow (1 - \eta_t \lambda)\, \mathbf{w} + \eta_t \mathbf{1}\Big[y_i \mathbf{w}^\top \mathbf{x}_i < 1\Big]\, y_i \mathbf{x}_i, \qquad (4)$$

optionally followed by a projection of $\mathbf{w}$ onto the ball of radius $1/\sqrt{\lambda}$. After $T$ steps its error in the objective shrinks like $1/(\lambda T)$, so small $\lambda$ needs many steps.

### C. Random Fourier features

A kernel SVM replaces $\mathbf{x}_i^\top \mathbf{x}_j$ by a kernel such as the RBF $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$, whose $n \times n$ Gram matrix is too large for our data. Rahimi and Recht [4] approximate it with an explicit map $\mathbf{z}(\mathbf{x}) = \sqrt{2/D}\cos(W^\top \mathbf{x} + \mathbf{a})$, with $W_{jk} \sim \mathcal{N}(0, 2\gamma)$ and $a_k \sim \mathcal{U}[0, 2\pi)$, so that $\mathbb{E}[\mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}')] = k(\mathbf{x}, \mathbf{x}')$. A linear SVM on $\mathbf{z}(\mathbf{x})$ then approximates the RBF-kernel SVM.

## III. Data and Preprocessing

The data is the Kaggle bina.az sale collection [6]: one CSV of 100,775 listings and 51 columns. All preprocessing is in `data_prep.py` (Table 1).

*Columns removed.* The pipeline drops 30 columns listed in `LEAKAGE_COLUMNS`. Only two are derived from the target: `unit_price`, the price per $m^2$, and `total_price`, which repeats the price; either would give the answer away. The others carry no information about the property or would identify listings and sellers: identifiers and URLs (`id_x`, `id_y`, `estate_id`, `estate_details_id_x/_y`, `rel_url`, `estate_rel_url`, `estate_rel_url_x/_y`, `img_url`); scrape and update times (`datetime_scrape_x/_y`, `day_x/_y`, `hour_x/_y`, `updated`, `city_when`); seller names (`owner_name`, `owner_title`, `shop_name`, `shop_title`); free text (`address`, `description`, `attributes`, `extra_info`); and the currency columns (`currency_x/_y`).

*Columns kept.* The remaining 21 columns are: the price (target); the location name (district, metro-station area or settlement) and city; floor area ($m^2$), number of rooms, floor ("floor / storeys"), land area (in sot); latitude, longitude and page views; category (new build, old build, house or villa, office), building type, renovation, title deed and mortgage availability (all originally in Azerbaijani); `products_label` (agency or residential complex); and five flag columns that hold a single value or are missing: `repair` (renovated), `bill_of_sale` (title deed available), `mortgage` (mortgage available), `vip` and `featured` (paid promotion of the listing).

*Cleaning*, in the order of `clean()`: (1) the floor string "*k* / *m*" is split at "/" into `floor` $= k$ and `total_floors` $= m$, unparsable or missing parts become 0, and `has_floor` is 1 when the string is present; (2) land area: "sot" and spaces are removed, a comma becomes a decimal point, and a missing value becomes 0; (3) price, area, rooms, latitude, longitude and views are converted to numbers after removing "$m^2$" and spaces and replacing commas by points, unparsable values becoming missing. There is no currency conversion: every raw row is priced in AZN and the currency columns are dropped; (4) rows missing price, area or rooms are removed (9,412 rows, all without a room count), then exact duplicate rows (4); missing values in seven text columns (location, city, building type, category, renovation, title deed, mortgage) become "Unknown"; (5) outliers: only prices of 10,000–5,000,000 AZN and areas of 10–2,000 $m^2$ are kept, removing 45 rows. This leaves 91,314 listings. No statistic of the data is computed during cleaning.

*Encoding.* `make_features` first turns the five flag columns into presence indicators, 1 if the column has a value and 0 if it is missing; one-hot encoding with `drop_first` would otherwise drop a single-valued column entirely. It then one-hot encodes every other text column with `pandas.get_dummies(drop_first=True)` and keeps the numeric columns as they are. This gives 171 features: 9 numeric, 5 presence flags and 157 indicators, 139 of them for location names. The flags are 1 for 85.5% of listings (`repair`), 77.8% (`bill_of_sale`), 34.1% (`mortgage`), 8.2% (`vip`) and 3.0% (`featured`). `repair`, `bill_of_sale` and `mortgage` are 1 on exactly the rows where the renovation, title-deed and mortgage columns say "yes", so they repeat information the one-hot indicators already carry; `vip` and `featured` add new information.

*Split and labels.* Every stage (EDA, SVM tuning, benchmark and error analysis) takes its split from one function, `get_splits` in `splits.py`. It calls `train_val_test_split` with a stratification label: the tier computed from the median of *all* prices. This label only keeps the premium share equal across the three splits; no model is trained or scored on it (Section X). The split is seeded and stratified at both steps: 15% for test, then 15/85 of the rest for validation. It yields 63,919 training, 13,697 validation and 13,698 test listings. `get_splits` then recomputes the labels the models use with `make_tier_label` on the training prices only: the threshold is the median *training* price, 215,000 AZN, a listing is premium (1) if its price is at least the threshold, and validation and test use the same threshold. The classes are balanced: 50.3% premium in each split. For the SVMs, `standardize` rescales every feature with the training mean and standard deviation (a constant column is left unscaled).

Table 1. Dataset summary. Features: numeric + presence flags + one-hot indicators.

| Property | Value |
|---|---|
| Raw rows / columns | 100,775 / 51 |
| Rows after cleaning | 91,314 |
| Features (see caption) | 171 (9 + 5 + 157) |
| Train / val / test rows | 63,919 / 13,697 / 13,698 |
| Tier threshold (train median) | 215,000 AZN |
| Premium %, train / val / test | 50.3 / 50.3 / 50.3 |

## IV. Exploratory Data Analysis

We explored the training split only (63,919 listings), so nothing seen here can leak validation or test prices into a choice. Prices are strongly right-skewed (Fig. 1): the mean is 271,961 AZN against a median of 215,000, and the skewness is 6.00. The logarithm removes most of it (skewness 0.28), which is why Task A predicts $\log(\text{price})$: squared errors on the log scale weigh a 10% error equally at every price level. The tiers have 31,786 standard and 32,133 premium training listings.

Floor area is the strongest single signal (Fig. 2): log area and log price correlate at 0.73. The median price rises with the number of rooms, from 105,500 AZN for one room to 510,000 AZN for six or more, but the price ranges of neighbouring room counts overlap widely. On the map, the most expensive listings form a dense cluster in central Baku, at the south-west of the plotted area, and prices fall towards the outer settlements. Location therefore matters beyond size, which the models confirm (Section VIII).

## V. Method

### A. Decision tree (from scratch)

*Structure.* Each `Node` stores a feature index and threshold (internal nodes), its two children, a leaf value, and the number of training rows and the impurity at the node. `fit` records the classes present in $y$ and grows the tree recursively from the root at depth 0. The `random_state` argument only seeds NumPy's global generator; the algorithm itself uses no randomness.

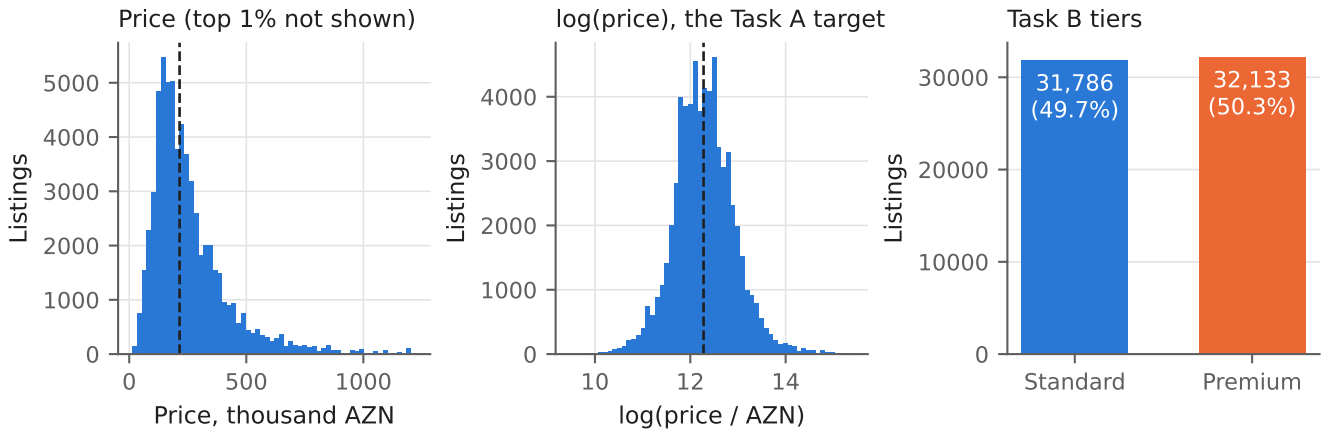



Figure 1. Training split. Left: price, without the 640 listings above the 99th percentile (1,230,740 AZN). Middle: log(price), the Task A target. Dashed: the tier threshold. Right: Task B class balance.

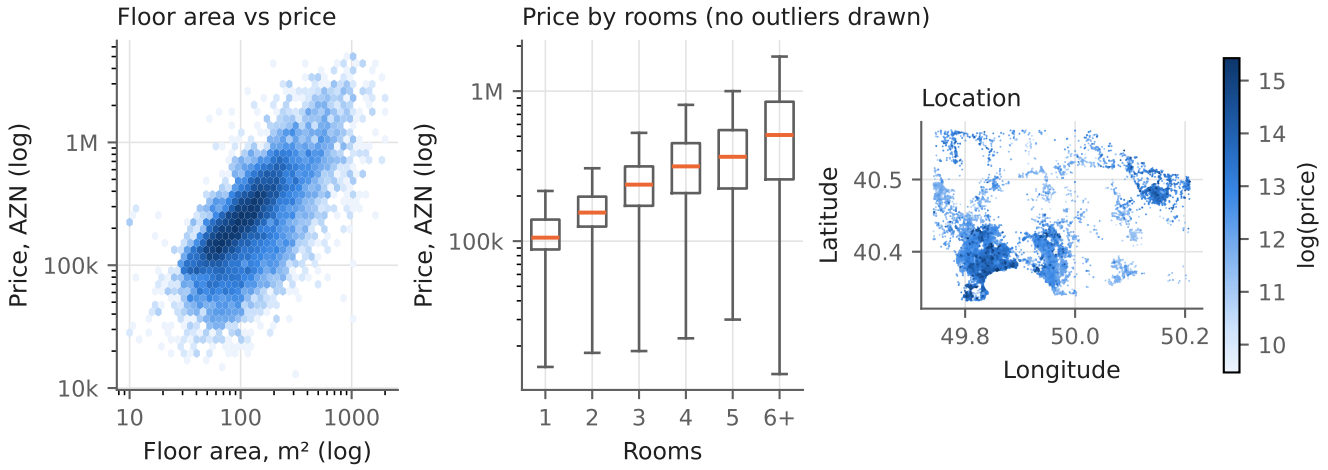



Figure 2. Training split. Left: floor area against price (log-log, darker = more listings). Middle: price by number of rooms (boxes: quartiles; whiskers: 1.5 IQR). Right: listing locations coloured by log(price); the map shows the central 98% of latitudes and longitudes, 61,593 of the 63,919 listings.

*Growth and stopping.* `_grow` first stores the node's row count and impurity. The node becomes a leaf if it is pure, has fewer than `min_samples_split` rows, or has reached `max_depth`; otherwise it searches for the best split, and becomes a leaf if none is valid. A valid split leaves at least `min_samples_leaf` rows in each child and decreases the impurity by at least `min_impurity_decrease`, measured as (2) at the node (scikit-learn additionally weights this test by the node's share of all rows). The default of this fourth parameter is 0, so a split with zero gain is still accepted. Rows with $x_j \le t$ go left, the others right.

*Split search.* For every feature, the candidate thresholds are the *observed* distinct values of that feature in the node (not midpoints between them, as in scikit-learn). For each candidate the children's impurities are recomputed from scratch, and the split with the largest decrease (2) wins; on a tie the first one found, i.e. the lowest feature index and smallest threshold, is kept. A node with $m$ rows thus costs $O(m)$ per candidate threshold, which makes the search slow (Section VIII).

*Criteria.* For classification the impurity is Gini or entropy with $\log_2$, as in (1). For regression it is the variance of the node's targets (`np.var`), i.e. $I_{\text{MSE}}$, whatever the `criterion` string.

*Prediction.* A regression leaf stores the mean target, a classification leaf the class frequencies of its rows. A sample follows the thresholds to a leaf; `predict` returns the mean, or the class with the highest frequency (a tie goes to the first class, standard), and `predict_proba` returns the frequencies. We use the premium frequency as the ROC score.

*Feature importance.* `tree_feature_importances` (in `evaluate.py`) walks the fitted tree and adds, for every internal node, its weighted impurity decrease $n_t I_t - n_L I_L - n_R I_R$ to the node's split feature, then normalises the sum to 1, using the stored node statistics. This is scikit-learn's mean decrease in impurity.

*Correctness check.* `tests/test_decision_tree.py` fits our tree and scikit-learn's with the same settings (`max_depth=2`) on two six-row toy datasets and requires identical class predictions and regression predictions equal to five decimals. Section VIII repeats the comparison on the full data.

### B. Support-vector machine (from scratch)

*Objective and labels.* `PegasosSVM` minimises (3) with an unregularised bias $b$. The two training classes are sorted and mapped to $-1$ and $+1$, so standard is $-1$ and premium $+1$; `predict` returns premium when the score $\mathbf{w}^\top \mathbf{z} + b \geq 0$.

*Update as implemented.* Training starts from $\mathbf{w} = \mathbf{0}$, $b = 0$ and makes one update per example; it is single-example, not mini-batch, Pegasos. With `n_epochs` set, as in all our runs, the data is visited in a new random order in each epoch, for $T = \texttt{n_epochs} \cdot n$ steps. At step $t$ the code computes the margin $y_i(\mathbf{w}^\top \mathbf{z}_i + b)$ with the current parameters, sets $\eta_t = 1/(\lambda t^p)$ with $p = 1$ (the default, used throughout), shrinks $\mathbf{w} \leftarrow (1 - \eta_t \lambda)\mathbf{w}$ and, if the margin is below 1, adds $\eta_t y_i \mathbf{z}_i$ to $\mathbf{w}$ and $\eta_t y_i$ to $b$. Finally $\mathbf{w}$ (not $b$) is projected onto the ball of radius $1/\sqrt{\lambda}$. This is update (4) plus a bias that takes the same sub-gradient step but is never shrunk or projected. The `batch_size` parameter only splits the Fourier mapping and prediction into blocks to bound memory.

*Diagnostics.* The model records the objective (3) and the mean hinge loss on all training rows at step 0, every `history_every` steps and at the last step; we set `history_every` to $n$, one checkpoint per epoch. After training it stores $\|\mathbf{w}\|$, the margin width $2/\|\mathbf{w}\|$, and as support vectors the training rows with $y_i(\mathbf{w}^\top \mathbf{z}_i + b) \leq 1 + 10^{-10}$, i.e. on or inside the margin, including misclassified rows. This margin-based definition differs from the non-zero dual coefficients of a QP solver.

*Fourier features.* The RFF variant uses the class `RandomFourierFeatures`, which draws $W$ and $\mathbf{a}$ as in Section II from a seeded generator, once, when the model is fit, and maps every input with $\mathbf{z}(\mathbf{x}) = \sqrt{2/D}\cos(\mathbf{x}W + \mathbf{a})$; Pegasos then learns $\mathbf{w} \in \mathbb{R}^D$ and $b$. We use $D = 500$. RFF was chosen as the non-linear option for memory: an exact float64 kernel matrix on our training split would need 32.7 GB, the 500-dimensional Fourier matrix 256 MB. The RFF model did not, however, win the validation comparison: the final SVM is linear (Section VII). Both variants are sweepable with the same code, `svm_experiment.sweep_svm` for RFF.

## VI. Experimental Setup

### A. Targets, metrics and scaling

Task A predicts log(price); we report RMSE, MAE and $R^2$ on the log scale and, after $\exp(\cdot)$, in AZN. For Task B we report precision, recall and F1 for the premium class and ROC-AUC computed from tree leaf probabilities or SVM decision values. All metrics are our own NumPy code, tested against `sklearn.metrics`. The SVMs receive standardized features; the trees use the unscaled features.

### B. Cross-validation and the held-out test

No choice uses the test split. Cross-validation uses a stratified sample of 20,000 of the 77,616 training and validation rows and 5-fold stratified K-fold. Inside every fold, the tier threshold and the SVM scaler are recomputed from that fold's training rows only, and every model sees the same folds and features. We report the mean $\pm$ sample standard deviation over folds. The subsample keeps the run practical: our tree takes about 10 s per fit at 16,000 rows and 87 s on the full training split. For the held-out test, each model is fit once on the full training split and scored once on the test split.

### C. Matched scikit-learn baselines

Our trees are compared with `DecisionTreeRegressor` and `DecisionTreeClassifier` (Gini and entropy) with the same `max_depth`, `min_samples_split`, `min_samples_leaf` and seed. Linear Pegasos is compared with `SGDClassifier` (hinge loss, L2 penalty, $\alpha = \lambda$, the same number of epochs), which minimises the same objective (3), and with `SVC(kernel="linear")`. RFF Pegasos is compared with the same random-feature construction (`RBFSampler`, same $\gamma$ and dimension) followed by `SGDClassifier`, and with `SVC(kernel="rbf")` at the same $\gamma$. For `SVC` we set $C = 1/(n\lambda)$, which makes its objective a rescaling of (3); $n$ is the number of rows SVC actually sees. Exact-kernel SVC grows roughly quadratically with $n$, so it is fit on at most 10,000 stratified training rows; all other models use every training row.

### D. Timing and seeds

We record wall-clock fit and prediction time for every model on one machine (Apple M4), and the fit time of both trees and both linear SVM solvers at 2,000, 4,000, 8,000 and 16,000 training rows. Every random choice (split, subsamples, folds, models, random features) uses the seed 42.

## VII. Hyperparameter Tuning

### A. SVM: regularisation and kernel width

Both SVMs were tuned on train $\rightarrow$ validation, fitting on all 63,919 training rows and scoring on the 13,697 validation rows. The grid has 7 values of $\lambda$, from $10^{-7}$ to $10^{-1}$, for the linear kernel and for RFF with $D = 500$ and $\gamma \in \{0.1, 1, 10\} \times 1/171$, where $1/171$ (one over the number of features) is the usual *scale* default for standardized inputs; each model trains for 20 epochs. The selection metric is validation F1; ROC-AUC is recorded too.

Fig. 3 shows the sweep. The linear SVM peaks at $\lambda = 10^{-3}$ with F1 0.904. Smaller $\lambda$ lets $\|\mathbf{w}\|$ grow to 2,716 at $10^{-7}$ and F1 fall to 0.866; larger $\lambda$ widens the margin (1.71 at $10^{-1}$, against 0.45 at $10^{-3}$) and puts 32,036 training rows on or inside it, underfitting at F1 0.860. RFF does best with small $\lambda$ and the widest kernel (smallest $\gamma$): its best F1 is 0.903, at $\gamma = 0.1 \times 1/171$ and $\lambda = 10^{-6}$, while $\gamma = 10 \times 1/171$ never exceeds 0.817. At $\lambda = 10^{-1}$ every RFF model scores F1 0.669, the F1 of labelling every validation listing premium.

*Selection rule and stability check.* We do not simply take the highest F1. Within each kernel, among configurations whose validation F1 is within 0.005 of the best, we take the *largest* $\lambda$. Pegasos uses the step size $1/(\lambda t)$ and its guarantee after $T$ updates scales as $1/(\lambda T)$, so it needs $T \gg 1/\lambda$, and a small $\lambda$ that wins on the full training split can be under-trained on less data. To test this, each kernel's highest-validation-F1 configuration and its chosen configuration are re-evaluated with the benchmark's cross-validation design: 20,000 training and validation rows, 5 folds, 16,000 training rows per fold and 20 epochs, i.e. 320,000 updates per fit. For the linear kernel both configurations are $\lambda = 10^{-3}$, and its validation F1 of 0.904 holds up in cross-validation

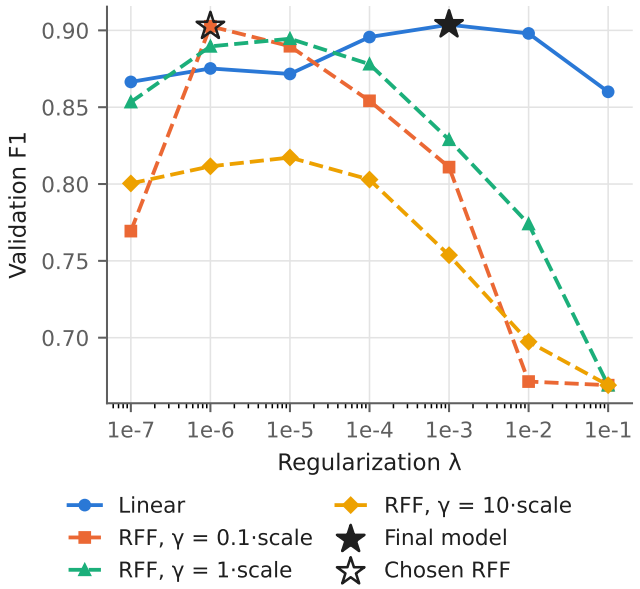



Figure 3. SVM tuning: validation F1 against $\lambda$ for the linear kernel and RFF at three kernel widths ($scale = 1/171$). Stars: the configuration chosen for each kernel; filled: the final model.

at $0.897 \pm 0.003$. For RFF both are $\lambda = 10^{-6}$ with $\gamma = 0.1 \times scale$: no configuration with a larger $\lambda$ came within 0.005 of its validation F1 of 0.903, so the rule did not change the choice. This configuration needs $1/\lambda = 10^6$ updates but gets 320,000 per fold, and its cross-validation F1 falls to $0.763 \pm 0.095$. The rule only guards against this when a larger $\lambda$ is nearly as good on validation, which was not the case for RFF on this split.

*Final SVM.* The final SVM is the kernel whose chosen configuration has the higher validation F1: linear, 0.904 against 0.903 for RFF, although RFF has the marginally higher validation ROC-AUC (0.9647 against 0.9637). Both SVMs are benchmarked.

### B. Decision tree

Both tasks use one fixed tree configuration, set before any experiment and not tuned: maximum depth 10, at least 10 rows to split a node, at least 5 rows per leaf, and no minimum impurity decrease. These are the fixed values in `benchmark.py`; the pipeline runs no tree sweep, so nothing overrides them. For Task B we compare the two criteria in cross-validation: Gini and entropy reach the same F1, $0.907 \pm 0.006$ and $0.907 \pm 0.004$, while entropy ranks better (ROC-AUC $0.962 \pm 0.004$ against $0.951 \pm 0.005$) and fits faster (9.1 s against 10.3 s per fold). We report the Gini tree as our main Task B tree because that choice was fixed in advance; on F1 the data gives no reason to prefer either.

## VIII. Results

Table 2 lists every model. On Task A our tree reaches a test $R^2$ of 0.853 on log price (0.753 in AZN), against 0.852 for the scikit-learn tree; in cross-validation the two score $0.831 \pm 0.006$ and $0.831 \pm 0.005$. The gap between log scale and AZN comes from the most expensive listings (Section IX).

On Task B the trees lead. Our Gini tree reaches test F1 0.926 and ROC-AUC 0.971, equal to scikit-learn's tree (F1 0.926). The final linear Pegasos SVM reaches F1 0.904 and ROC-AUC 0.962, level with its baselines: SGD 0.905 and linear SVC 0.904. RFF Pegasos gets F1 0.900, above RFF SGD (0.887) and below exact RBF SVC (0.907). Each of our models is within 0.007 F1 of its best matched baseline; the largest gap is RFF Pegasos against the exact RBF SVC. Trained on the full split, RFF Pegasos is competitive, but it is the least stable model in cross-validation, as the stability check predicted (Section VII): F1 $0.763 \pm 0.095$, against $0.897 \pm 0.003$ for linear Pegasos. SGD on the same random features is also unstable ($0.829 \pm 0.047$), while the exact RBF SVC is not ($0.903 \pm 0.001$). Fig. 4 shows that the ROC curves of our models and their baselines nearly coincide.

*Our trees versus scikit-learn's.* The trees are practically the same model. Both Task B Gini trees have depth 10, 464 and 462 leaves, split first on floor area, and agree on 99.9% of test labels. The root thresholds differ only by convention: 90.1 $m^2$ for ours, an observed training value, and 90.15 $m^2$ for scikit-learn, the midpoint between neighbouring values. The Task A trees both have 656 leaves; their predictions correlate at 0.9989, differ by 0.0015 log units on average, and 99.6% agree within 0.01 log units (about 1% of the price). The feature importances correlate above 0.9999 for all three pairs (Fig. 5). Floor area dominates both tasks, followed by longitude and latitude, in line with the EDA.

*Speed.* Matching the models' quality costs time for the tree. On the full training split our Gini tree fits in 86.8 s against 0.31 s for scikit-learn, about 277 times slower. Fig. 6 shows the growth with $n$: from 2,000 to 16,000 rows our tree's fit time grows 15.0-fold for 8 times the data, scikit-learn's 10.4-fold, and the two linear SVM solvers 8.6-fold and 9.3-fold, close to linearly. Linear Pegasos fits the full training split in 2.7 s, against 0.29 s for SGD.

## IX. Error Analysis

We analyse the test errors of our Task A tree and, on Task B, of our Gini tree and the final linear SVM.

*The tier boundary.* Task B errors are roughly balanced between the classes (precision 0.921, recall 0.930 for the tree; Fig. 7) and concentrate near the threshold. For listings within 5% of 215,000 AZN the tree misclassifies 32.3% and the SVM 42.8%, while beyond 40% from the threshold they err on 1.3% and 1.5%. The 5% band alone holds 28% of all tree errors. Most of the remaining Task B error is therefore noise in listing prices close to an arbitrary cut-off.

*Price extremes.* On Task A the median absolute percentage error is 14.3% in the cheapest decile and 19.6% in the most expensive, against 9.3–12.5% for deciles 2–9 (Fig. 8). The errors are biased towards the middle: the cheapest decile is over-predicted by a median 7.8% and the most expensive under-predicted by 13.8%. A tree of depth 10 predicts leaf means, so prices at the extremes are pulled towards the average of their leaf. The effect is strongest for the 138 most expensive listings (top 1%, from 1,300,000 AZN): median absolute error 33.9% and a median under-prediction of 33.6%. For Task B the same listings are easy (tree error 0.0%), being far from the threshold.

*Sparse districts.* Task A error grows as a district's training data shrinks. Grouping test listings by how many *training*

Table 2. Headline results on the held-out test set, with the 5-fold CV mean ± std where marked "CV". Task A errors are on the log scale unless marked AZN; Task B metrics are for the premium class. n/a: the model is for the other task. *Fit on at most 10,000 training rows. All metrics: Tables 3 and 4.

| | Task A: log price | | | | Task B: premium tier | | |
|---|---|---|---|---|---|---|---|
| Model | RMSE (log) | $R^2$ (log) | $R^2$ (log), CV | $R^2$ (AZN) | F1 | F1, CV | ROC-AUC |
| Tree (ours) | 0.241 | 0.853 | $0.831 \pm 0.006$ | 0.753 | n/a | n/a | n/a |
| Tree (sklearn) | 0.242 | 0.852 | $0.831 \pm 0.005$ | 0.739 | n/a | n/a | n/a |
| Tree gini (ours) | n/a | n/a | n/a | n/a | 0.926 | $0.907 \pm 0.006$ | 0.971 |
| Tree gini (sklearn) | n/a | n/a | n/a | n/a | 0.926 | $0.907 \pm 0.006$ | 0.971 |
| Tree entropy (ours) | n/a | n/a | n/a | n/a | 0.920 | $0.907 \pm 0.004$ | 0.975 |
| Tree entropy (sklearn) | n/a | n/a | n/a | n/a | 0.920 | $0.909 \pm 0.004$ | 0.975 |
| Pegasos linear (ours) | n/a | n/a | n/a | n/a | 0.904 | $0.897 \pm 0.003$ | 0.962 |
| SGD hinge linear (sklearn) | n/a | n/a | n/a | n/a | 0.905 | $0.894 \pm 0.004$ | 0.962 |
| SVC linear (sklearn)* | n/a | n/a | n/a | n/a | 0.904 | $0.903 \pm 0.002$ | 0.960 |
| Pegasos RFF (ours) | n/a | n/a | n/a | n/a | 0.900 | $0.763 \pm 0.095$ | 0.963 |
| SGD hinge RFF (sklearn) | n/a | n/a | n/a | n/a | 0.887 | $0.829 \pm 0.047$ | 0.964 |
| SVC RBF (sklearn)* | n/a | n/a | n/a | n/a | 0.907 | $0.903 \pm 0.001$ | 0.965 |

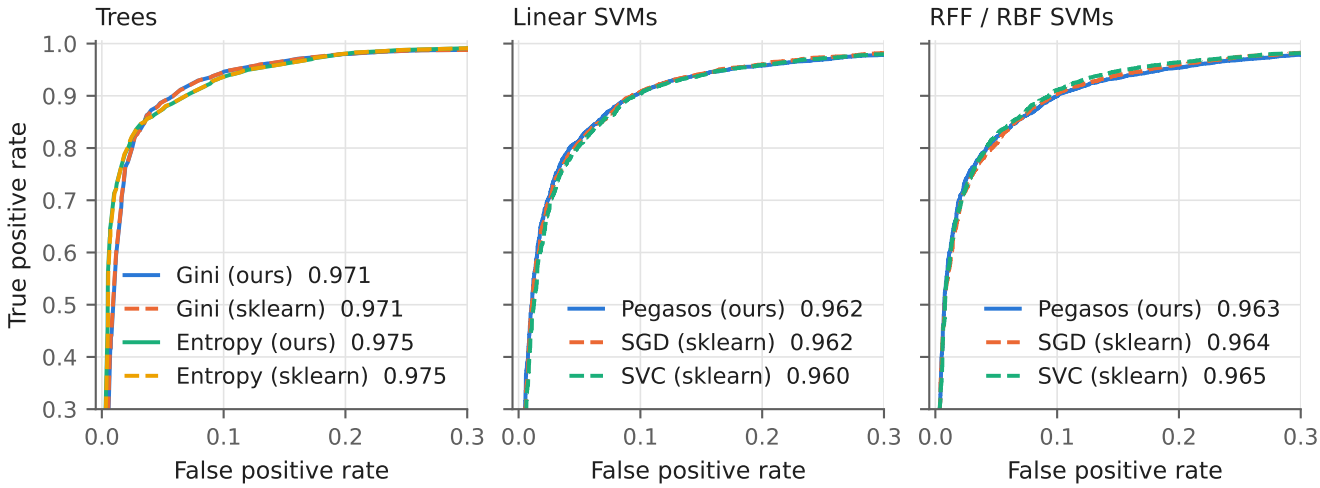



Figure 4. Task B ROC curves on the test set by model family, zoomed on false-positive rates up to 0.3. Solid: our models; dashed: scikit-learn. Legend numbers: ROC-AUC. SVC is fit on at most 10,000 rows.

listings their district has, the median absolute error is 11.4% for districts with 1,000 or more training rows, 13.1% for 200–999, 18.6% for 50–199 and 20.0% for 1–49. Task B shows no such trend: the tree's error rate is 6.9% in the largest districts, 11.8% for 50–199 and 5.8% for 1–49, whose listings are mostly cheap (15.7% premium) and therefore far from the threshold. One test listing comes from a district with no training rows.

*Categories.* Houses and villas are the hardest large group, with a median absolute error of 19.1% against 11.3% for new-build and 10.3% for old-build apartments. Offices score 34.4%, but there are only 25 of them in the test set.

*Worst listings.* Nine of the ten worst Task A listings are houses or villas whose price differs from our prediction by a factor of 4.8 to 8.7 in either direction. Some look like data-entry errors rather than model failures, such as a seven-room house with 11 $m^2$ of floor area on 310 sot, listed at 280,000 AZN. No model can price such listings, and they inflate RMSE in AZN.

*What the SVM uses.* The final linear SVM has $\|\mathbf{w}\| = 4.49$, a margin $2/\|\mathbf{w}\| = 0.445$ in standardized units, and 17,946 of the 63,919 training points on or inside the margin. Floor area carries by far the largest weight towards premium; being in an outlying settlement of Baku carries the largest weights towards standard (all weights: `results/error_analysis/svm_linear_weights.csv`).

## X. Limitations

- *The trees were not tuned.* Both tasks use the fixed configuration in `src/benchmark.py` (maximum depth 10, minimum 10 samples to split, minimum 5 per leaf), because the pipeline contains no tree sweep on the real data, while the SVMs were tuned on validation. The trees' results are therefore not their best, and the tuning effort between the two model families is unequal.
- *Cross-validation uses a subsample.* CV uses 20,000 of the 77,616 available rows, so each fold trains on 16,000 rows, a quarter of the full training split. CV scores are correspondingly lower than the test scores (Task A $R^2$ 0.831 against 0.853) and describe the variability of a smaller-data model; for RFF Pegasos the smaller folds are enough to break training (Section VII).

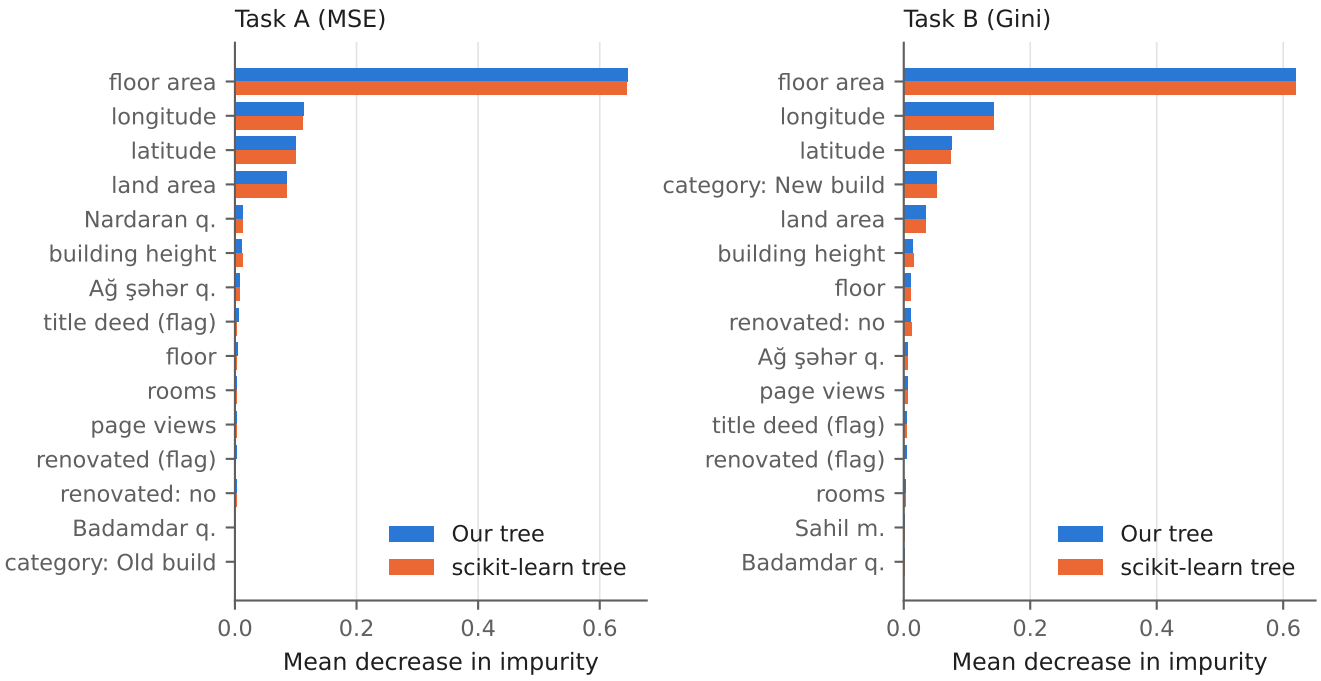



Figure 5. Feature importance (mean decrease in impurity, normalised) of our trees and scikit-learn's: the 15 most important features by our importance. Location names end in q. (settlement), m. (metro-station area) or r. (district).

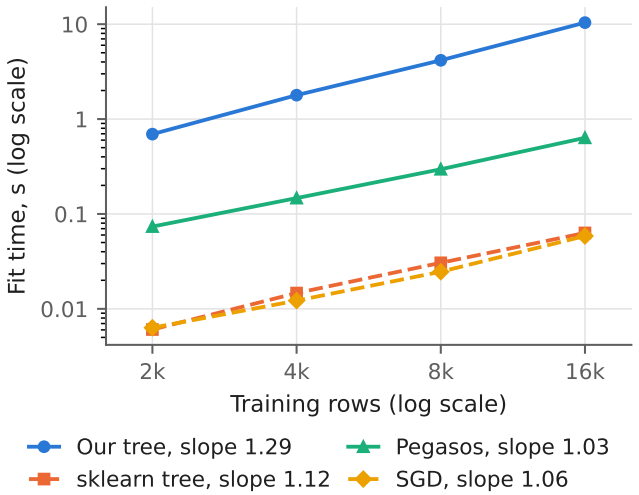



Figure 6. Fit time against training size (log-log), Task B, one fit per point. Slope: fitted exponent $k$ in time $\propto n^k$. Our tree grows faster than linearly; both SVM solvers grow close to linearly.

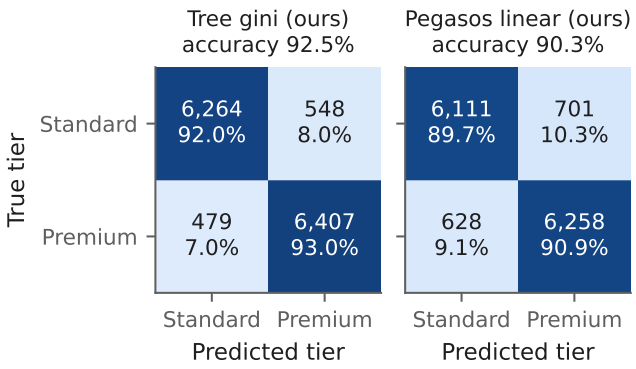



Figure 7. Task B confusion matrices on the test set, our Gini tree and our final linear Pegasos SVM (counts and share of the true tier).

- *One-hot categories come from all rows.* The encoding is built before the split, so categories that occur only in validation or test still get a column. Two such columns are all-zero in training. No statistic leaks, and the models never use these columns, but a strict pipeline would fit the encoder on training rows only.

- *Stratification on a full-data label.* `get_splits` balances the three splits with a tier computed from the median of *all* prices, test prices included, so which listings land in the test split depends on that median. No model is trained or scored on this label: the labels the models use come from the training median. On this data both medians are 215,000 AZN, so the two labelings differ on 0 listings.

- *Our tree is slow.* It is about 277 times slower than scikit-learn on the full training split and grows faster than linearly with $n$ (Fig. 6). For every candidate threshold it recomputes the impurity from scratch instead of sweeping sorted values with running sums. This forced the CV subsample and left no budget to tune the tree.

- *Two fields are filled with 0.* For the 15,338 listings without a floor (15,188 houses and 150 offices), floor and building height are set to 0, with a `has_floor` flag; for the 76,126 listings without a land area, the land area is set to 0. Here 0 means "not applicable", not a measured zero. The trees can separate the two through the flag and the category, but the linear SVM treats 0 as a real value.




Figure 8. Errors by true-price decile (decile edges from training prices), test set. Left: Task A median absolute percentage error of our tree. Right: Task B error rate; the threshold lies between deciles 5 and 6.

## XI. Conclusion

Our from-scratch decision tree and Pegasos SVM reach the accuracy of matched scikit-learn models on bina.az listings: each is within 0.007 F1 of its best baseline, and our trees reproduce scikit-learn's almost split for split. The tree is the better model on both tasks, with test $R^2$ 0.853 on log price and F1 0.926 on the price tier, and floor area and location explain most of its decisions. On validation the linear SVM narrowly beat the Fourier-feature SVM, and only the linear SVM is stable when trained on less data: the RFF configuration with the best validation F1 needs more Pegasos updates than a cross-validation fold provides. The remaining errors sit at the tier threshold, at the price extremes and in districts with few listings, and a few come from implausible listings. The price of writing the tree ourselves is speed: it fits about 277 times more slowly. Future work: ensembles of our trees (bagging or boosting), a vectorised split search over sorted values with running sums, which would also make a proper tree sweep affordable, and better location features, such as distance to the city centre or target statistics of each district computed on training folds.

## Author Contributions

Mahammad Mammadli implemented the data pipeline (`data_prep.py`): loading, cleaning, parsing of floor and land area, leakage removal, encoding, the stratified split and the training-only tier threshold. Ibrahim Aghazade implemented the from-scratch decision tree (`decision_tree.py`) with Gini, entropy and MSE criteria, the four stopping parameters, node statistics and the scikit-learn match test; the tree hyperparameter sweep was not completed, so the final tree uses a fixed configuration. Sahib Aliyev implemented the Pegasos SVM (`svm.py`) with margin, support vectors and random Fourier features, plus the sweep and plotting tools (`svm_experiment.py`, `svm_plots.py`) and their tests. Khizri Sakhrayev implemented the NumPy metrics and tree feature importances (`evaluate.py`), the scikit-learn benchmark (`benchmark.py`), the SVM tuning on the real data with the stability-based selection rule (`svm_tuning.py`), the error analysis (`analysis.py`), the EDA figures (`eda.py`), the one-command pipeline (`run_all.py`) and the README, fixed the encoding of the five binary flag columns in the data pipeline, and wrote and assembled the report.

## Tools and Acknowledgements

Khizri Sakhrayev used Claude Code (Anthropic) to produce `evaluate.py`, `tests/test_evaluate.py`, `benchmark.py`, `svm_tuning.py`, `analysis.py`, `eda.py`, `splits.py`, `tests/test_splits.py`, `run_all.py`, the README and the text of this report, under his direction and review. Sahib Aliyev used Codex for the SVM implementation, random Fourier features, tuning and plotting helpers, tests and implementation documentation, as recorded in `docs/SVM.md`. We used NumPy, pandas, matplotlib and scikit-learn [5]; scikit-learn only for the baselines and tests.

## References

[1] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Belmont, CA: Wadsworth, 1984.

[2] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[3] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[4] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems 20*, 2007, pp. 1177–1184.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] sehriyarmemmedli (Kaggle user), "binaaz-sale-project," Kaggle dataset, 2025. `https://www.kaggle.com/datasets/sehriyarmemmedli/binaaz-sale-project`

## A. Full Results

Tables 3 and 4 give every metric of every model: the held-out test value and, below it, the 5-fold CV mean ± std. AZN errors are in thousands of AZN. *Fit on at most 10,000 training rows.

Table 3. Task A (regression of log(price)), all metrics.

| Model | Eval. | RMSE (log) | MAE (log) | $R^2$ (log) | RMSE (kAZN) | MAE (kAZN) | $R^2$ (AZN) |
|---|---|---|---|---|---|---|---|
| Tree (ours) | test | 0.241 | 0.166 | 0.853 | 124.9 | 51.5 | 0.753 |
| | CV | $0.257 \pm 0.006$ | $0.172 \pm 0.003$ | $0.831 \pm 0.006$ | $139.6 \pm 11.1$ | $54.3 \pm 1.2$ | $0.674 \pm 0.037$ |
| Tree (sklearn) | test | 0.242 | 0.166 | 0.852 | 128.4 | 51.7 | 0.739 |
| | CV | $0.256 \pm 0.005$ | $0.172 \pm 0.003$ | $0.831 \pm 0.005$ | $140.8 \pm 12.1$ | $54.3 \pm 1.0$ | $0.669 \pm 0.034$ |

Table 4. Task B (premium tier), all metrics for the premium class.

| Model | Eval. | Precision | Recall | F1 | ROC-AUC |
|---|---|---|---|---|---|
| Tree gini (ours) | test | 0.921 | 0.930 | 0.926 | 0.971 |
| | CV | $0.906 \pm 0.003$ | $0.908 \pm 0.012$ | $0.907 \pm 0.006$ | $0.951 \pm 0.005$ |
| Tree gini (sklearn) | test | 0.921 | 0.930 | 0.926 | 0.971 |
| | CV | $0.906 \pm 0.004$ | $0.908 \pm 0.013$ | $0.907 \pm 0.006$ | $0.953 \pm 0.004$ |
| Tree entropy (ours) | test | 0.912 | 0.928 | 0.920 | 0.975 |
| | CV | $0.906 \pm 0.009$ | $0.909 \pm 0.006$ | $0.907 \pm 0.004$ | $0.962 \pm 0.004$ |
| Tree entropy (sklearn) | test | 0.912 | 0.928 | 0.920 | 0.975 |
| | CV | $0.908 \pm 0.011$ | $0.910 \pm 0.007$ | $0.909 \pm 0.004$ | $0.963 \pm 0.003$ |
| Pegasos linear (ours) | test | 0.899 | 0.909 | 0.904 | 0.962 |
| | CV | $0.893 \pm 0.006$ | $0.901 \pm 0.011$ | $0.897 \pm 0.003$ | $0.956 \pm 0.003$ |
| SGD hinge linear (sklearn) | test | 0.900 | 0.910 | 0.905 | 0.962 |
| | CV | $0.887 \pm 0.010$ | $0.902 \pm 0.007$ | $0.894 \pm 0.004$ | $0.955 \pm 0.005$ |
| SVC linear (sklearn)* | test | 0.895 | 0.914 | 0.904 | 0.960 |
| | CV | $0.898 \pm 0.007$ | $0.907 \pm 0.009$ | $0.903 \pm 0.002$ | $0.959 \pm 0.002$ |
| Pegasos RFF (ours) | test | 0.874 | 0.928 | 0.900 | 0.963 |
| | CV | $0.945 \pm 0.064$ | $0.664 \pm 0.183$ | $0.763 \pm 0.095$ | $0.957 \pm 0.002$ |
| SGD hinge RFF (sklearn) | test | 0.930 | 0.848 | 0.887 | 0.964 |
| | CV | $0.947 \pm 0.022$ | $0.742 \pm 0.091$ | $0.829 \pm 0.047$ | $0.957 \pm 0.002$ |
| SVC RBF (sklearn)* | test | 0.897 | 0.916 | 0.907 | 0.965 |
| | CV | $0.900 \pm 0.007$ | $0.907 \pm 0.009$ | $0.903 \pm 0.001$ | $0.963 \pm 0.003$ |